Fullstack + QA Automation Assessment

As part of our interview process, we have included a test automation assessment. Please automate and solve the questions mentioned below, then push your solution to GitHub and share the link. Include test results as well.

Read these instructions carefully:

- Build a Playwright framework, not a plain test script — use separate feature, step definition, and page (Page Object Model) files, with environment configuration (no hardcoded URLs).
- Use dynamic, resilient locators — avoid brittle selectors (positional CSS/XPath) that break when a parent element changes. Prefer role/label/text-based or data-testid locators.
- AI self-healing exercise:
  - Add 3–5 incorrect/brittle locators in your test files (leave them broken).
  - Add a markdown file describing how you'd self-heal these locators using AI — detection, prompt approach, and validation before applying the fix. A working POC is a bonus.

Using Claude Code (or any other AI tools)

We expect you to use Claude Code (or a similar AI tool) throughout this project — not just to scaffold the initial code, but as a genuine pair programmer. In practice, that looks like:

- Start by describing the framework you want to build in plain language and let it scaffold the folder structure, page objects, step definitions, and config.
- Iterate — ask it to fix bugs, add scenarios, explain unfamiliar Playwright/Cucumber APIs or SQL patterns.
- Notice when it's wrong and correct it — hallucinated selectors/APIs, bad locator strategies, and subtle logic bugs are common.
- Use it to move faster through unfamiliar territory (e.g. self-healing locators, window functions for the SQL streak scenario), not to replace your own judgment on what's actually correct.

Claude Code reflection — include a short section in your README covering how you used it, what worked, and what did not.

Submission (applies to everything below)

- Publish all code to GitHub with a clear README (setup instructions, how to run, architecture notes).
- The repository must include your test execution results (report / screenshots / logs) — not just the test code.

© 2026 Streamhub LTD. All rights reserved.

streamhub

Please complete EITHER Section A OR Section B, as per your choice.

You do not need to complete both sections.

Section A: Web Application + UI Automation

A1. Build a web application

Build a small web application of your own. You may build something in the spirit of the EMI Calculator (an interactive calculator/tool https://emicalculator.net/), or keep it generic — for example, a web app with a dashboard and reports section. At minimum, your app should include:

- A dashboard or landing view showing some summarized/calculated data.
- At least one report or detail view driven by user input (e.g. filters, a form, or interactive controls).
- At least one chart or visual element (e.g. pie chart, bar chart, or table) reflecting the underlying data.

A2. Automate it with Playwright

Write Playwright test cases for the web app you built above, following the framework requirements at the top of this document (feature/step/page files, dynamic locators, environment config). Cover, at minimum:

- Navigating to the dashboard/report view and validating it loads correctly.
- Submitting input and validating the calculated/displayed output matches your own independently computed expected value.
- Verifying the visibility of the chart/visual element and that it renders non-zero, valid data.

Execute the tests and include the results (HTML report, screenshots, or console output) in your GitHub repo.

A3. API Test (Automate using Playwright)

Target: JSONPlaceholder

Objective: Validate the API's handling of boundary and invalid data during post creation.

Endpoint: POST https://jsonplaceholder.typicode.com/posts

Steps:

- Send a POST request containing:
  - Excessively long strings for titles.
  - Unsupported special characters.
  - Missing required fields (e.g., userId).
- Expected Outcome: The API should respond with the appropriate HTTP error codes or error messages for invalid inputs without encountering server-side failures.

© 2026 Streamhub LTD. All rights reserved.

A4. SQL Tests

Please write SQL queries for the scenarios listed below. You may create these databases locally or use an online SQL compiler. Provide the table schema and screenshots of your query outputs.

Scenario 1: Find instances where Account A sends money to Account B, and Account B sends a similar amount (within 10%) back to Account A. Both transactions must happen within a 24-hour window. This test identifies quick "round-trip" transfers or potential payment reversals.

Scenario 2: IPL Player Performance Streaks — Using an IPL-style dataset for the 2024 season, identify players who scored 30+ runs in at least 3 consecutive matches. Return the player's name and the date the scoring streak commenced.

Section B: API Development + API Automation

B1. Build an API

Build a small API of your own that returns data from a mock database or JSON files (no real DB connection required). Your API should:

- Expose at least 2–3 endpoints.
- Support various query parameters (e.g. filtering, sorting, pagination, or search by field).
- Return appropriate status codes and error messages for invalid parameters.

B2. Automate it with Playwright

Write Playwright API test cases for the API you built above, following the framework requirements at the top of this document. Cover, at minimum:

- Happy-path requests for each endpoint and parameter combination.
- Invalid/edge-case parameters (e.g. out-of-range values, unsupported types, missing required parameters).
- Correct status codes and response shape for both valid and invalid requests.

Execute the tests and include the results (HTML report, screenshots, or console output) in your GitHub repo.

B3. UI Test Cases (EMI Calculator)

Playwright, API, and SQL

Tools/Software Required: Node.js, Playwright, and Cucumber

Application: https://emicalculator.net/

Test Case 1: Validate the EMI Pie Chart (please automate all steps mentioned below using playwright and cucumber)

Steps:

- Launch the application URL mentioned above.
- Navigate to the Home Loan tab.

© 2026 Streamhub LTD. All rights reserved.

- Input the following values into the application. Calculate the EMI within your code and validate that your calculated output matches the figures displayed in the application:
  - Scenario A: Home Loan Amount: 25L, Interest Rate: 10%, Tenure: 10 Years
  - Scenario B: Home Loan Amount: 50L, Interest Rate: 7.5%, Tenure: 15 Years
- Verify the visibility and availability of the pie chart.
- Extract the numerical values from both sections of the pie chart. The step should Pass if the values are greater than zero; otherwise, it should Fail.

Test Case 2: Validate the EMI Bar Chart

Steps:

- Launch the application URL mentioned above.
- Navigate to the Personal Loan tab.
- Interact with the sliders for "Personal Loan Amount," "Interest Rate," and "Loan Tenure" to set the following values:
  - Scenario: Personal Loan Amount: 10L, Interest Rate: 12%, Tenure: 5 Years
- Modify the month using the "Schedule showing EMI payments starting from" calendar widget.
- Verify the visibility and availability of the bar chart.
- Count the total number of bars available in the chart.
- Retrieve and validate the values displayed in the tooltip of any single bar.

B4. SQL Tests

Please write SQL queries for the scenarios listed below. You may create these databases locally or use an online SQL compiler. Provide the table schema and screenshots of your query outputs.

Scenario 1: Find instances where Account A sends money to Account B, and Account B sends a similar amount (within 10%) back to Account A. Both transactions must happen within a 24-hour window. This test identifies quick "round-trip" transfers or potential payment reversals.

Scenario 2: IPL Player Performance Streaks — Using an IPL-style dataset for the 2024 season, identify players who scored 30+ runs in at least 3 consecutive matches. Return the player’s name and the date the scoring streak commenced.

© 2026 Streamhub LTD. All rights reserved.